Faculdade de Ciências da Universidade de Lisboa

Secure Gallery Log System

José Sampaio, fc66191
Rodrigo Neto, fc59850
Gabriel Cambraia, 58671

Segurança de Redes e Sistemas Distribuídos
2026

Contents

1 Introduction **2**

2 System Architecture **2**
2.1 Overview 2
2.2 Threat Model 2
2.3 Design Decisions 2
2.3.1 Known Limitations 3

3 Log Format **3**
3.1 Plaintext Entry Format 3
3.2 On-Disk Record Layout 3

4 Security Protocol **4**
4.1 Cryptographic Primitives 4
4.2 Keystream Generation 4
4.3 Protocol Descriptions — Append and Verification 5

5 Implementation Details **6**
5.1 logappend 6
5.2 logread 6

6 Performance Analysis **6**
6.1 Empirical Measurements 7
6.2 A Possible Optimisation: Encrypted State Snapshot 7

7 Security Analysis **8**
7.1 Attack 1: Tail Truncation and Rollback to an Older Version 8
7.2 Attack 2: Record Reordering and Insertion 8
7.3 Attack 3: Symlink / File-Injection Attack 9
7.4 Attack 4: TOCTOU Race / Concurrent Append 9

8 Intentional Vulnerability **9**

1 Introduction

For this first phase of the project we built two CLI tools, `logappend` and `logread`, that together manage an append-only, encrypted gallery log. The log tracks employee and guest arrivals and departures through a gallery and its rooms: `logappend` creates the log and writes new events to it, while `logread` answers queries about it — the current state, which rooms a given person has visited, or which rooms two people shared at the same time. Security is the main design concern: the system should preserve the confidentiality, integrity, and authenticity of every log entry, while still being fast enough for interactive use. We picked Rust because it gives us memory safety without giving up low-level control or performance.

2 System Architecture

2.1 Overview

The system consists of two CLI tools: `logappend`, which writes new events to the log after validating the full chain and gallery state machine, and `logread`, which reads and decrypts the log to answer state, room-history, and intersection queries. Both tools share a single library crate (`lib.rs`) that contains all crypto, encoding, state-machine, and query logic. The log itself is stored as a single flat binary file, with no external database or index.

2.2 Threat Model

We assume the adversary has full read/write access to the log file on disk but does *not* know the authentication token. Under this model, the properties we aim to guarantee are: confidentiality of all log contents, so that no information about entries is disclosed to an unauthorized party; integrity, ensuring no forging, reordering, insertion, or silent deletion of entries goes undetected; authenticity, so that only a holder of the correct token can append or read the log; and freshness, preventing the replay of old commands. We consider denial-of-service attacks out of scope, since an adversary with write access can always simply delete the file. Side-channel attacks on the running process, such as timing analysis of MAC verification or inference from record lengths, are likewise not addressed.

2.3 Design Decisions

Key management is kept simple by deriving a single key from the authentication token: one SHA-256 derivation produces the 32-byte key used for both encryption and MAC computation. This means an incorrect token immediately produces a wrong key, causing the first MAC check to fail and preventing any unauthorized access to the log's contents. Every record is encrypted with a stream cipher before it hits the disk, then authenticated with HMAC-SHA-256 under the Encrypt-then-MAC construction, so the MAC is verified before any plaintext is recovered and tampered ciphertext is rejected without ever being decrypted. To enforce ordering and prevent insertion or reordering, we use a hash chain: each record's ciphertext is hashed, and that hash is embedded in the next record's plaintext as both a chain link and a unique nonce for the stream cipher. Any modification to the sequence, whether reordering, inserting, or silently deleting records in the middle, breaks the chain and is detected on the next read. Chaining also keeps

writes incremental: an append only needs to encrypt and authenticate the new record rather than rewriting the whole file, though reads still have to walk the full chain to validate integrity (see 2.3.1). Logical consistency is enforced through a state machine that replays every entry in sequence. The state machine rejects impossible transitions such as double arrivals, departures from the wrong room, or leaving when not present, and requires strictly increasing timestamps to prevent replay of old commands. This design means that every append requires a full-chain validation, resulting in $O(n)$ reads per operation. We accept this trade-off because it guarantees both integrity and state consistency on every write. Batch mode amortises the cost by validating the chain once and then appending the entries incrementally.

2.3.1 Known Limitations

Log truncation and rollback to an older file version remain undetectable, because there is no out-of-band anchor for the current chain tip. Fixing this properly would need a companion checkpoint file or an external counter, neither of which we implemented for this phase. Token leakage through command-line arguments and process memory is inherited from the specification's requirement to pass the token via a `-K` flag and is accepted as a known limitation. Additionally, while the token is hashed through SHA-256 before use, no password-stretching algorithm such as Argon2 or PBKDF2 is applied, meaning a weak token could be brute-forced offline by MAC-checking candidates against the first record.

3 Log Format

3.1 Plaintext Entry Format

Before encryption, each entry is encoded as a pipe-delimited ASCII string followed by 32 raw bytes corresponding to the hash of the previous record to build the chain.

`ts|type|name|act|room`	`prev_hash`

Figure 1: Plaintext entry layout

The text portion encodes five pipe-separated fields: the `timestamp` as a decimal integer, the `person_type` as either `E` (employee) or `G` (guest), the `name` as an ASCII string, the `action` as either `A` (arrival) or `L` (departure), and the `room` as a decimal number or simply as an empty string for gallery level events. The final 32 bytes contain the raw SHA-256 hash of the previous record's ciphertext, which serves as the hash-chain link. The first record uses 32 zero bytes (`0x00...00`) as the genesis anchor.

3.2 On-Disk Record Layout

Each record on disk is composed of three contiguous fields: the length prefix, the variable-length ciphertext, and a fixed-size MAC tag. Records are concatenated back-to-back with no file header, padding or delimiters.

Len	Ciphertext	MAC

Figure 2: On-disk record layout

The **Length** field (4 bytes) is a big-endian `u32` encoding the size of the ciphertext in bytes. The **Ciphertext** contains the encrypted plaintext entry and varies in size depending on the input. The **MAC** (final 32 bytes) is an HMAC-SHA-256 tag computed over the ciphertext, following the Encrypt-then-MAC construction. Each record therefore carries a fixed overhead of 36 bytes (4 bytes for the length prefix and 32 bytes for the MAC) on top of the ciphertext.

4 Security Protocol

4.1 Cryptographic Primitives

The entire cryptographic scheme is built from a single primitive: SHA-256. Key derivation consists of a single-pass SHA-256 over the ASCII token, producing the 32-byte key K used for both encryption and MAC computation. Encryption uses a custom CTR-mode stream cipher where each keystream block i is computed as SHA-256($K\|$nonce$\|i$), with the nonce set to the previous record's `prev_hash` (32 bytes) and i encoded as a 64-bit little-endian counter. This is behaviorally equivalent to AES-CTR but uses SHA-256 as the pseudorandom function. Message authentication uses standard HMAC-SHA-256 over the ciphertext following Encrypt-then-MAC, so any tampered record is caught before the stream cipher is ever invoked. Finally, the chain hash is computed as SHA-256 of each record's ciphertext, which becomes the `prev_hash` for the next record, creating a hash chain analogous to a Merkle chain where each record commits to the full history before it.

Primitive	**Construction**	**Output size**
Key derivation	SHA-256(token)	32 bytes
Stream cipher	SHA-256($K\|$nonce$\|$ctr)	32 bytes/block
MAC	HMAC-SHA-256(K, ciphertext)	32 bytes
Chain hash	SHA-256(ciphertext)	32 bytes

Table 1: Cryptographic primitives and their parameters

4.2 Keystream Generation

The stream cipher generates keystream in 32-byte blocks by evaluating SHA-256($K\|$nonce$\|i$) for successive counter values $i = 0, 1, 2, \ldots$ The plaintext is XOR'd with the concatenated keystream byte-by-byte. Since plaintext lengths are not generally a multiple of 32, the final block is truncated: only the bytes needed to cover the remaining plaintext are used, and the rest of the keystream block is discarded.

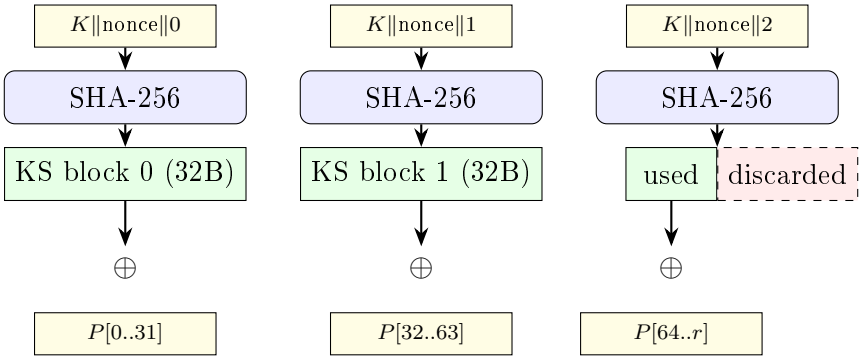
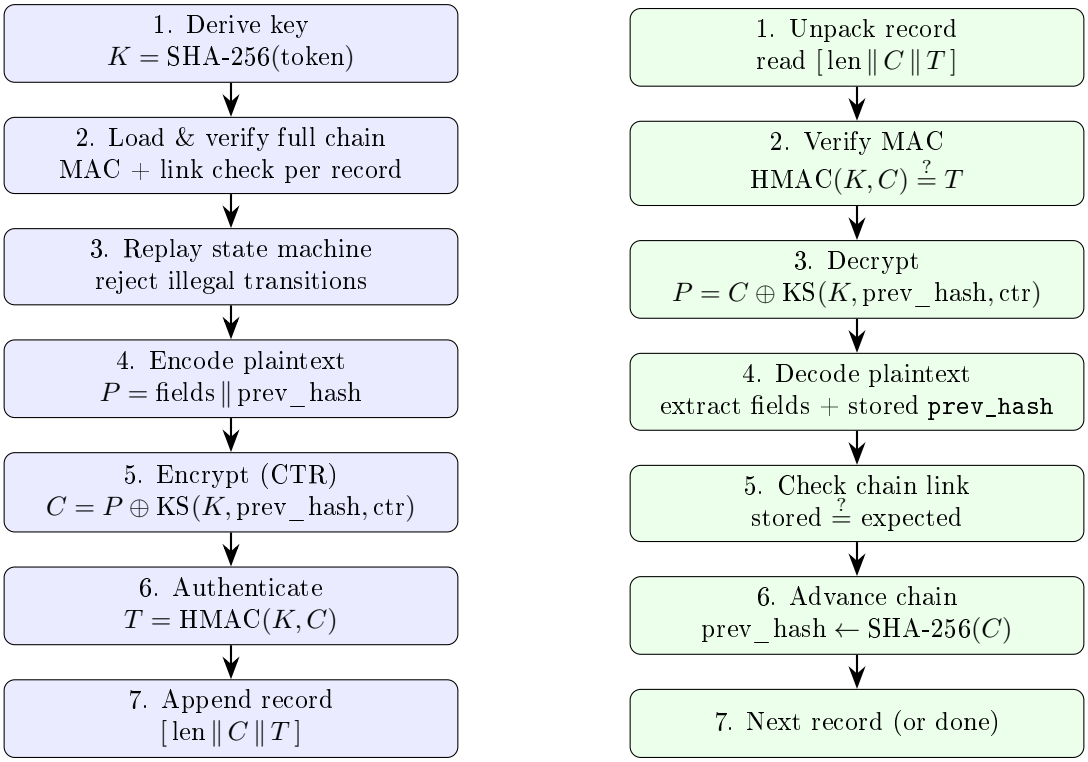


Figure 3: Keystream generation in CTR mode. Each block produces 32 bytes of keystream via SHA-256. The final block is truncated to match the remaining plaintext length; unused bytes are discarded.

Within a single log, nonce reuse is prevented by construction: every record beyond the first uses a `prev_hash` derived from the previous record's ciphertext, so no two records share the same nonce and keystreams are never repeated. The genesis record is a deliberate exception — it uses the all-zero anchor as its nonce, which is discussed in §8.

4.3 Protocol Descriptions — Append and Verification



(a) Append flow

(b) Verification (load) flow

Figure 4: Append and verification protocol flows

5 Implementation Details

5.1 logappend

The `logappend` tool uses a manual flag-based argument parser that validates the token (alphanumeric), the name (alphabetic), the timestamp (> 0), and the mutual exclusivity of `-E`/`-G` and `-A`/`-L` before any file access occurs. Once the arguments are validated, it acquires an exclusive advisory lock (`flock(LOCK_EX)`) on a companion `.lock` file via `libc::flock()` and holds it for the entire load-validate-append critical section, which prevents TOCTOU races between concurrent appenders. All file opens use `O_NOFOLLOW` (set through `OpenOptionsExt::custom_flags`) so that symlinked paths are rejected with `ELOOP` rather than silently followed.

With the lock held, the full chain is loaded and verified, after which every entry is replayed through a gallery state machine that enforces strictly increasing timestamps, forbids duplicate gallery arrivals, requires gallery presence before any room entry, rejects simultaneous occupancy of multiple rooms, and matches each room departure against the current room. Batch mode amortises this work: the chain is loaded and validated once, the resulting `(state, last_hash)` pair is cached across lines that share the same log path and key, and each subsequent line is applied and appended incrementally in $O(1)$ instead of $O(n)$, with the lock held for the full batch duration.

5.2 logread

The `logread` tool mirrors the same locking discipline, but on the read side: it acquires a shared lock (`flock(LOCK_SH)`) on the companion `.lock` file, which lets multiple readers proceed concurrently while still blocking during an active write. Every invocation performs a full chain validation, where each record goes through MAC check $\rightarrow$ decrypt $\rightarrow$ chain check, and any mismatch causes the read to be rejected. Three query types are supported: `-S` reports the current gallery state (who is in the gallery and which rooms they occupy), `-R` returns the ordered list of rooms a given person has entered, and `-I` computes the intersection of rooms where all specified persons were simultaneously present, using interval overlap detection across the target persons on a per-room basis.

6 Performance Analysis

Because every append must recover `last_hash` and verify integrity before it can write, a single append runs in $O(n)$, where n is the current number of records. Batch mode amortises this: for k lines on the same log, the chain is validated once and each subsequent append is $O(1)$ (encode, encrypt, MAC, write), giving an overall cost of $O(n + k)$. Read and query operations are also $O(n)$, since the full chain must be decrypted and validated regardless of which query is issued.

On disk, each record carries a fixed overhead of 36 bytes — a 4-byte length prefix and a 32-byte MAC — on top of the ciphertext, and the ciphertext preserves the plaintext length since we use a stream cipher. A typical entry with a short name weighs in at roughly 50–70 bytes of plaintext, which translates to about 86–106 bytes once serialised. One deliberate optimisation is that `prev_hash` is stored as the raw 32 bytes rather than a 64-byte hex encoding, saving 32 bytes per record — around a 30% reduction in plaintext size for short entries.

The main bottleneck is the full-chain validation that runs on every single append and on every read. We measure its practical cost below and then sketch an optimisation that would remove it from the append and `-S` paths.

Operation	**Cost**
Single append	$O(n)$
Batch append (k lines, same log)	$O(n + k)$
Read / query (`-S`, `-R`, `-I`)	$O(n)$
Per-record on-disk overhead	36 bytes (4B len + 32B MAC)

Table 2: Asymptotic cost and storage overhead per operation, where n is the number of records in the log.

6.1 Empirical Measurements

To back up the asymptotic analysis with real numbers, we wrote a benchmark suite (`tests/perf_test.rs`, with a helper Python script `tests/gen_perf_batch.py` that produces valid batch files of arbitrary size). Each configuration starts from a fresh log, appends N entries (the $N = 1$ case through a direct `logappend` call, the rest through `logappend -B`), and then runs the three read queries against the resulting log: `-S` for the current state, `-R -E Alice` for Alice's room history, and `-I -E Alice -E Bob` for the rooms Alice and Bob shared. Every configuration is repeated a few times and the median wall-clock time is reported. All numbers come from the `release` build on a local Linux machine.

Mode	$\boldsymbol{N}$	**Append (ms)**	**Per-line (ms)**	`-S` **(ms)**	`-R` **(ms)**	`-I` **(ms)**
single	1	1.49	1.4855	1.38	1.47	1.50
batch	5	1.66	0.3312	1.38	2.02	2.26
batch	50	2.62	0.0524	2.22	2.27	2.34
batch	500	8.85	0.0177	3.55	2.70	3.53
batch	5 000	77.67	0.0155	8.48	14.05	14.56
batch	50 000	682.78	0.0137	77.62	72.31	74.84

Table 3: Wall-clock cost of an append-plus-three-reads workload (medians, release binary). The *Per-line* column divides total append time by N.

Batch mode amortises all the fixed-cost work (process start-up, argument parsing, key derivation, lock acquisition, initial full-chain validation, final `GalleryState` construction) across the whole batch, so the per-line append cost drops from about 1.5 ms at $N = 1$ to roughly 14 μs at $N = 50\,000$ — a two-orders-of-magnitude improvement.

6.2 A Possible Optimisation: Encrypted State Snapshot

An earlier version of this report alluded to storing a hash of the whole log file to skip revalidation on append, but that idea does not really help: even if integrity is trusted, the append still has to read and decrypt every record to rebuild the current `GalleryState` before the state machine can validate the new entry. The version of the idea that *does* remove the bottleneck is to persist, alongside the log, an encrypted and authenticated

snapshot of the `GalleryState` itself, together with the `last_hash` of the log tail at the moment it was written, all protected under the same key K. An append then just loads the snapshot, checks that its embedded `last_hash` still matches the log's actual tail, applies the new entry to the cached state, and writes both the new record and the updated snapshot — cold append becomes $O(1)$, and so does the `-S` query. `-R` and `-I` still need the full event history, so they would not benefit without a richer snapshot. If the embedded tail hash ever disagrees with the file, the snapshot is discarded and we fall back to a full re-validation, so the optimisation degrades gracefully. Full rollback of both files in lockstep remains undetectable without an external trust anchor, as discussed in Attack 2.

7 Security Analysis

7.1 Attack 1: Tail Truncation and Rollback to an Older Version

Description: An adversary with write access either removes the last k records from the log file or replaces the whole file with an older valid version from a backup or snapshot. Both actions produce a file that is a valid prefix of some past chain, so `logread` walks it without error — the most recent events are silently erased or reverted.
Root cause: There is no out-of-band anchor for the "current tip" of the chain, so the tip is whatever the file itself says it is. The hash chain binds every record to its predecessor, but nothing binds the last record to the fact that it is really the last.
Mitigation status: *Not mitigated in this phase.* A proper fix needs an external trust anchor that the attacker cannot rewrite in lockstep with the log: for example a companion state snapshot (see §6.2) stored on an append-only medium, a monotonic counter kept in a different trust domain (`chattr +a` file, separate volume, remote store), or a TPM. We considered a plain companion checkpoint file but rejected it because an adversary with write access to both files can simply roll them back together.
Code reference: `load_log()` at `src/lib.rs:150-177` walks records until `off >= buf.len()` and treats the last one it sees as the tip, with no external check against the file having been shortened or reverted.

7.2 Attack 2: Record Reordering and Insertion

Description: An adversary with write access tries to reorder existing records on disk (for example, swapping two legitimate records to make it look as though Alice arrived in room 1 before Bob left it), or to splice a brand-new fabricated record into the middle of the log.
Root cause: Records are concatenated back-to-back with no positional metadata, so the physical byte order is the only thing that dictates the logical order. Without a binding between adjacent records, both reordering and insertion would go unnoticed.
Mitigation status: **Fully mitigated** by the combination of the hash chain and the per-record MAC. Every plaintext embeds the SHA-256 of the previous record's ciphertext as a built-in link, and the verifier recomputes the expected link from the running tail hash on each step, so any reordering immediately breaks the embedded `prev_hash` check. Forging a new record to splice into the middle additionally requires producing a valid HMAC-SHA-256 tag under the token-derived key, which the attacker does not have. Together, the two defences mean no undetected reordering and no forged insertion.

Code reference: The chain link is appended to every plaintext in `LogEntry::encode()` at `src/lib.rs:83-91` (raw 32 bytes, no hex encoding), `append_entry()` at `src/lib.rs:181-193` writes the encrypted record plus its MAC tag, and the verification loop in `load_log()` at `src/lib.rs:165-173` runs *MAC check → decrypt → stored_prev vs expected* on every record; the stored-link test at `src/lib.rs:170` is the specific line that rejects reordered and spliced records.

7.3 Attack 3: Symlink / File-Injection Attack

Description: If the log path is a symlink to a sensitive file (e.g., `/etc/passwd`), a default `OpenOptions::create(true).append(true)` would follow the symlink and append ciphertext to the target. The practical impact is low because what ends up on the target is opaque binary data, but it is still a symlink-follow weakness that could corrupt an unrelated file.
Root cause: `File::open` and `OpenOptions::open` follow symlinks by default on Unix.
Mitigation status: Fully mitigated. All file opens use `O_NOFOLLOW` via `OpenOptionsExt::custom_fl` if the target path is a symlink, the open fails with `ELOOP` and the operation is rejected before any byte is read or written.
Code reference: `load_log()` at `src/lib.rs:154-157` and `append_entry()` at `src/lib.rs:186-190`, both of which chain `.custom_flags(libc::O_NOFOLLOW)` onto their `OpenOptions` before calling `.open()`.

7.4 Attack 4: TOCTOU Race / Concurrent Append

Description: Two concurrent `logappend` processes both call `load_log`, both compute the same `prev_hash`, and both append. The second writer's record carries a `prev_hash` that no longer matches its actual predecessor on disk, so the chain breaks on the next read; in the worst case, interleaved `write_all` calls can splice two records' bytes together and corrupt the file.
Root cause: Without explicit file locking, `O_APPEND` is only atomic per single `write(2)` below `PIPE_BUF` on Linux, and that guarantee is not portable to other Unixes.
Mitigation status: Fully mitigated. An exclusive advisory lock (`libc::flock(fd, LOCK_EX)`) on a companion `.lock` file wraps the entire load → validate → append critical section in `logappend`, while `logread` uses a shared lock (`LOCK_SH`) so concurrent reads are allowed but blocked during an active write.
Code reference: `lock_log_exclusive()` at `src/lib.rs:365-377` and `lock_log_shared()` at `src/lib.rs:380-392` implement the primitives; they are acquired by `run_single()` at `src/bin/logappend.rs:104`, `run_batch()` at `src/bin/logappend.rs:158`, and `main()` at `src/bin/logread.rs:103`, in each case held across the full load/validate/append (or load/query) section.

8 Intentional Vulnerability

Description. The keystream for each record is derived as SHA-256($K\|$prev_hash$\|$ctr). For every log, the very first record uses the genesis anchor (prev_hash $= 0^{32}$) as its nonce. If two logs are created under the same token, they therefore share the same key K *and* the same nonce for their first record, which means both first records are encrypted under identical keystreams. XOR'ing the two first-record ciphertexts cancels the keystream and directly yields $P_1 \oplus P_2$.

Why it is subtle. Nonce uniqueness holds for every record beyond the first, since each subsequent nonce is the SHA-256 of the previous record's ciphertext and therefore depends on all data written so far. The failure is isolated to record 1, where no prior ciphertext exists and the nonce collapses to a fixed constant. A quick reading of the code — which passes an apparently "unique" `prev_hash` as the nonce — can easily miss that the genesis case defeats the uniqueness argument.

Potential exploitation. Token reuse across logs is a realistic threat: operators routinely recycle credentials across environments for convenience. Once two first-record ciphertexts are obtained, the attacker computes $C_1 \oplus C_2 = P_1 \oplus P_2$. Because the plaintext format is highly structured (`ts|type|name|act|room` followed by 32 zero bytes of `prev_hash`), most byte positions differ only in small, guessable substrings: the timestamp, the person type (`E`/`G`), the name, and the action (`A`/`L`). The trailing 32 bytes of `prev_hash` are identical in both plaintexts and so cancel out entirely in $P_1 \oplus P_2$, providing no information but also no obstacle. Guessing one plaintext (for example, one of the names) immediately recovers the corresponding portion of the other via $P_2 = P_1 \oplus (C_1 \oplus C_2)$.